
Lecture 7:

Structured Query Language (SQL) -

Data Definition Language (DDL) & Data Manipulation

Language (DML)

Updated: September 2025

Learning Outcomes

At the end of this lecture, students should be able to:

- ❑ **Explain** the purpose of SQL and its main components.
- ❑ **Differentiate** between SQL Data Definition Language (DDL) and SQL Data Manipulation Language (DML).
- ❑ **Create and modify** database tables using SQL DDL commands.
- ❑ **Insert, update, and delete** data using SQL DML commands.

Note: Data retrieval using the **SELECT statement** will be covered in **Lectures 8 to 11**.

Overview of SQL

- ❑ **SQL:** Standard language to manage and manipulate relational databases.
- ❑ **SQL Command Categories:**
 - **SQL Data Definition Language (DDL)** – Define/modify database structures (CREATE, ALTER, DROP).
 - **SQL Data Manipulation Language (DML)** – Used to manipulate data in tables (INSERT, UPDATE, DELETE, and SELECT).
 - **SQL Data Control Language (DCL)** – Control user access and permissions (GRANT, REVOKE).
 - **SQL Transaction Control Language (TCL)** – Manage transactions (COMMIT, ROLLBACK, SAVEPOINT).

Note: This lecture covers **only** SQL DDL and SQL DML.

❑ Key Features of SQL:

- **Simplicity** – The basic SQL command set uses fewer than 100 words, making it relatively easy to learn and use.
- **Standardization** – SQL is standardized by the American National Standards Institute (ANSI), ensuring a common structure and syntax.
- **Variations** – Many database systems, such as Oracle, MySQL, SQL Server, and PostgreSQL, implement SQL slightly differently, adding proprietary extensions or variations.
- **Set-Based Language**– SQL is designed for set-based operations, allowing you to work with multiple rows of data at once efficiently.

Part 1: SQL Data Definition Language (DDL)

- ❑ SQL DDL are used to define and create the **database structure** and its **components**.
- ❑ Key Steps in **Creating a Database**:
 - **Create Database Structure**
 - RDBMS creates **physical files** to store the database.
 - Implementation **varies between RDBMSs**.
 - Includes defining **storage**, file **locations**, and initial **configuration**.
 - **Create Tables**
 - Tables are created to **hold end-user data**.
 - Each table defines **columns**, **data types**, and **constraints**.
- ❑ **Authentication**:
 - Only **registered users** can access the database.
 - Users **log in** using a **user ID** and **password** set by the database administrator.

❑ SQL Data Definition Commands:

**TABLE
5.1**

SQL Data Definition Commands

COMMAND OR OPTION	DESCRIPTION
CREATE SCHEMA AUTHORIZATION	Creates a database schema
CREATE TABLE	Creates a new table in the user's database schema
NOT NULL	Ensures that a column will not have null values
UNIQUE	Ensures that a column will not have duplicate values
PRIMARY KEY	Defines a primary key for a table
FOREIGN KEY	Defines a foreign key for a table
DEFAULT	Defines a default value for a column (when no value is given)
CHECK	Validates data in an attribute
CREATE INDEX	Creates an index for a table
CREATE VIEW	Creates a dynamic subset of rows and columns from one or more tables (see Chapter 6, Procedural Language SQL and Advanced SQL)
ALTER TABLE	Modifies a table's definition (adds, modifies, or deletes attributes or constraints)
CREATE TABLE AS	Creates a new table based on a query in the user's database schema
DROP TABLE	Permanently deletes a table (and its data)
DROP INDEX	Permanently deletes an index
DROP VIEW	Permanently deletes a view

❑ SQL Data Definition Data Types:

- Data type selection is usually determined by the **nature of the data** and its **intended use**.
- Commonly supported SQL **data types** include:
 - **Numeric** – NUMBER(L,D), INTEGER, SMALLINT, DECIMAL(L,D)
 - **Character** – CHAR(L), VARCHAR(L), VARCHAR2(L)
 - **Date/Time** – DATE, TIME, TIMESTAMP
 - **Floating-point** – REAL, DOUBLE, FLOAT
 - **Intervals** – INTERVAL DAY TO HOUR

Note: Many **other specialized data types** may be supported depending on the database system.

❑ SQL Data Definition Data Types (cont.):

**TABLE
5.4**

Some Common SQL Data Types

DATA TYPE	FORMAT	COMMENTS
Numeric	NUMBER(L,D)	The declaration NUMBER(7,2) indicates that numbers will be stored with two decimal places and may be up to seven digits long, including the sign and the decimal place (for example, 12.32 or -134.99).
	INTEGER	May be abbreviated as INT. Integers are (whole) counting numbers, so they cannot be used if you want to store numbers that require decimal places.
	SMALLINT	Like INTEGER but limited to integer values up to six digits. If your integer values are relatively small, use SMALLINT instead of INT.
	DECIMAL(L,D)	Like the NUMBER specification, but the storage length is a <i>minimum</i> specification. That is, greater lengths are acceptable, but smaller ones are not. DECIMAL(9,2), DECIMAL(9), and DECIMAL are all acceptable.
Character	CHAR(L)	Fixed-length character data for up to 255 characters. If you store strings that are not as long as the CHAR parameter value, the remaining spaces are left unused. Therefore, if you specify CHAR(25), strings such as <i>Smith</i> and <i>Katzenjammer</i> are each stored as 25 characters. However, a U.S. area code is always three digits long, so CHAR(3) would be appropriate if you wanted to store such codes.
	VARCHAR(L) or VARCHAR2(L)	Variable-length character data. The designation VARCHAR2(25) will let you store characters up to 25 characters long. However, VARCHAR will not leave unused spaces. Oracle automatically converts VARCHAR to VARCHAR2.
Date	DATE	Stores dates in the Julian date format.

1. SQL DDL: Creating Database Tables

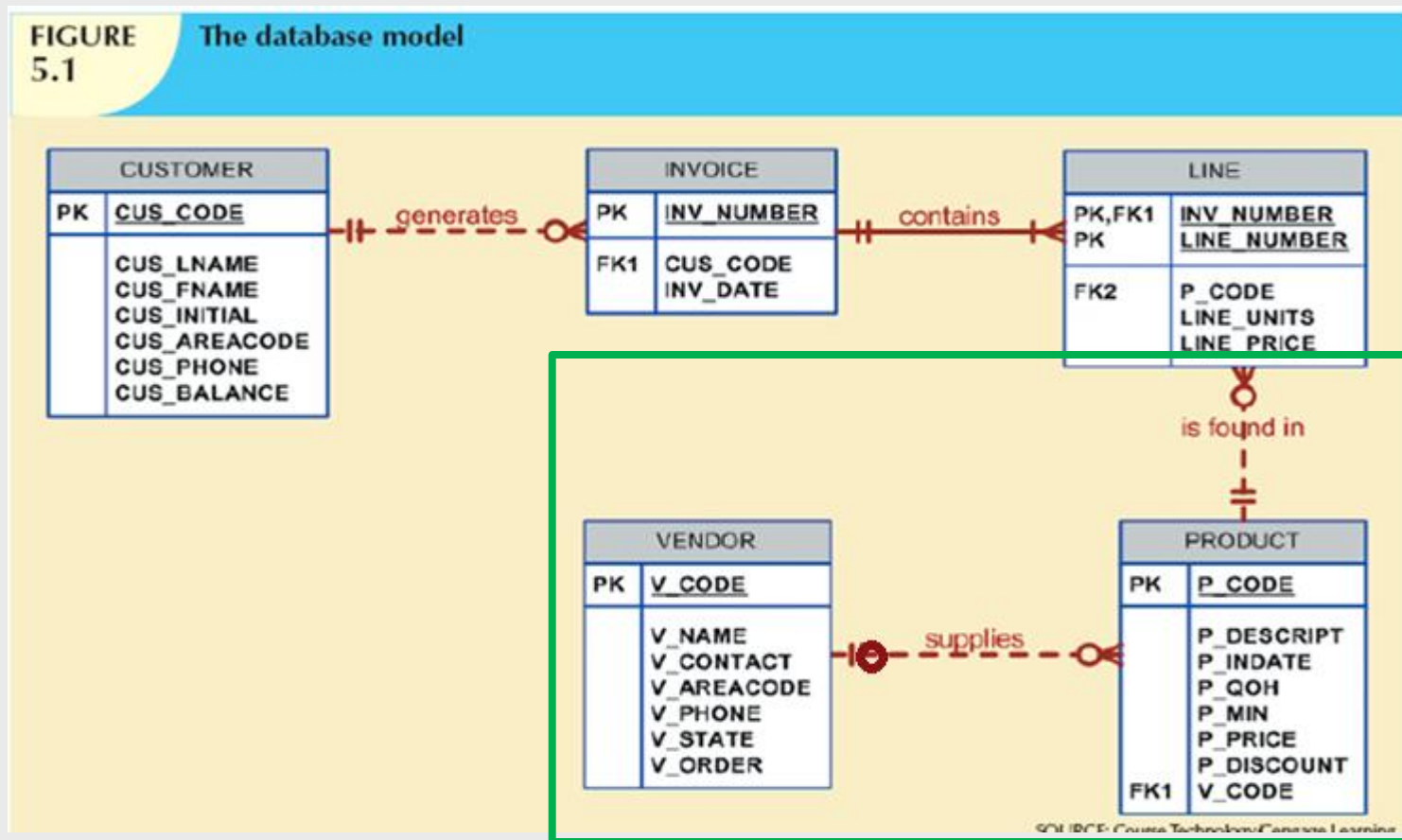
- ❑ **Column Definitions** – Use one line per column (attribute) definition.
- ❑ **Formatting** – Use spaces to align attribute characteristics and constraints for readability.
- ❑ **Naming Conventions** – Table and attribute names are typically capitalized.
- ❑ **Constraints** – Rules applied to table columns to enforce data integrity and consistency in a relational database.
- ❑ **Referential Integrity** – The RDBMS automatically enforces foreign key constraints.
- ❑ **Command Termination** – SQL statements end with a semicolon (;).

❑ SQL Constraints:

- **NOT NULL** – Ensures that a column **cannot contain null values**.
- **NULL** – Ensures that a column **can accept null values**.
- **UNIQUE** – Ensures that **all values in a column are distinct**.
- **PRIMARY KEY** – Uniquely identifies each row in a table; automatically **NOT NULL** and **UNIQUE**.
- **FOREIGN KEY** – Ensures **referential integrity** by linking to a primary key in another table
- **DEFAULT** – Assigns a **default value** to a column when a new row is inserted, if no value is provided.
- **CHECK** – Validates column values based on a **specified condition**.

-
- ❑ This lecture uses a **simple database** to illustrate SQL commands.
 - ❑ The database contains the following **tables**:
 - CUSTOMER
 - INVOICE
 - LINE
 - PRODUCT
 - VENDOR
 - ❑ The focus for this lecture is on the **PRODUCT** and **VENDOR** tables.

❑ The database model:



❑ The data dictionaries:

TABLE 5.3 Data Dictionary for the Ch05_SaleCo Database

TABLE NAME	ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE*	REQUIRED	PK OR FK	FK REFERENCED TABLE
VENDOR	V_CODE	Vendor code	INTEGER	#####	1000-9999	Y	PK	
	V_NAME	Vendor name	VARCHAR(35)	Xxxxxxxxxxxxxxx	NA	Y		
	V_CONTACT	Contact person	VARCHAR(25)	Xxxxxxxxxxxxxxx	NA	Y		
	V_AREACODE	Area code	CHAR(3)	999	NA	Y		
	V_PHONE	Phone number	CHAR(8)	999-9999	NA	Y		
	V_STATE	State	CHAR(2)	XX	NA	Y		
	V_ORDER	Previous order	CHAR(1)	X	Y or N	Y		
PRODUCT	P_CODE	Product code	VARCHAR(10)	XXXXXXXXXX	NA	Y	PK	
	P_DESCRIPT	Product description	VARCHAR(35)	Xxxxxxxxxxxxxxx	NA	Y		
	P_INDATE	Stocking date	DATE	DD-MON-YYYY	NA	Y		
	P_QOH	Units available	SMALLINT	####	0-9999	Y		
	P_MIN	Minimum units	SMALLINT	####	0-9999	Y		
	P_PRICE	Product price	NUMBER(8,2)	####.##	0.00-9999.00	Y		
	P_DISCOUNT	Discount rate	NUMBER(5,2)	0.##	0.00-0.20	Y		
	V_CODE	Vendor code	INTEGER	###	100-999		FK	VENDOR

❑ Syntax of the CREATE TABLE Statement:

```
CREATE TABLE table_name
(
    column_name_1 data_type column_constraint,
    column_name_2 data_type column_constraint,
    ...
    table_constraint
);
```

❑ Explanation of Syntax:

▪ Column Constraint style:

- A rule applied directly to a **single column** to control the kind of data it can store.
- Ensures data integrity for that **specific column**.

- **Table Constraint style:**

- A rule applied to the table as a **whole**, often involving **one or more columns**.
- Enforces **relationships** or **multi-column restrictions**.
- **Common Table Constraints:**
 - ✓ **PRIMARY KEY (col1, col2)** – Composite primary key.
 - ✓ **FOREIGN KEY (col_name) REFERENCES OtherTable(col_name)** – Ensures referential integrity.
 - ✓ **UNIQUE (col1, col2)** – Combination of columns must be unique.
 - ✓ **CHECK (condition)** – Validates a condition across multiple columns.

❑ **VENDOR Table Creation (Option 1 – Column Constraint Style):**

```
CREATE TABLE VENDOR
(
    V_CODE          INTEGER    PRIMARY KEY CHECK
                        (V_CODE>=1000 AND V_CODE<=9999) ,
    V_NAME          VARCHAR(35) NOT NULL,
    V_CONTACT       VARCHAR(25) NOT NULL,
    V_AREACODE      CHAR(3)    NOT NULL,
    V_PHONE         CHAR(8)    NOT NULL,
    V_STATE         CHAR(2)    NOT NULL,
    V_ORDER         CHAR(1)    NOT NULL
                        CHECK (V_ORDER='Y' OR V_ORDER='N')
);
```

❑ **VENDOR Table Creation (Option 2 – Table Constraint Style):**

```
CREATE TABLE VENDOR
(
    V_CODE          INTEGER,
    V_NAME          VARCHAR(35)    NOT NULL,
    V_CONTACT       VARCHAR(25)    NOT NULL,
    V_AREACODE      CHAR(3)        NOT NULL,
    V_PHONE         CHAR(8)        NOT NULL,
    V_STATE         CHAR(2)        NOT NULL,
    V_ORDER         CHAR(1)        NOT NULL,

    PRIMARY KEY (V_CODE) ,
    CHECK (V_CODE>=1000 AND V_CODE<=9999) ,
    CHECK (V_ORDER='Y' OR V_ORDER='N')
);
```

-
- ❑ After creating a table, use the **DESCRIBE** command (Oracle/MySQL) to **view its structure**. For example:

```
DESCRIBE VENDOR;  
DESC VENDOR;
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default
VENDOR	V_CODE	NUMBER	22	-	0	1	-	-
	V_NAME	VARCHAR2	35	-	-	-	-	-
	V_CONTACT	VARCHAR2	25	-	-	-	-	-
	V_AREACODE	CHAR	3	-	-	-	-	-
	V_PHONE	CHAR	8	-	-	-	-	-
	V_STATE	CHAR	2	-	-	-	-	-
	V_ORDER	CHAR	1	-	-	-	-	-

❑ **PRODUCT Table Creation (Option 1 – Column Constraint Style):**

```
CREATE TABLE PRODUCT (  
    P_CODE          VARCHAR(10)      PRIMARY KEY,  
    P_DESCRIPT      VARCHAR(35)      NOT NULL,  
    P_INDATE        DATE             NOT NULL,  
    P_QOH           SMALLINT         NOT NULL  
        CHECK (P_QOH >=0 AND P_QOH <= 9999),  
    P_MIN           SMALLINT         NOT NULL  
        CHECK (P_MIN >=0 AND P_MIN <= 9999),  
    P_PRICE         NUMBER(8,2)      NOT NULL  
        CHECK (P_PRICE >=0.00 AND P_PRICE <= 9999.00),  
    P_DISCOUNT     NUMBER(5,2)      NOT NULL  
        CHECK (P_DISCOUNT>=0.00 AND P_DISCOUNT<=0.20),  
    V_CODE          NULL            REFERENCES VENDOR (V_CODE));
```

❑ **PRODUCT Table Creation (Option 2 – Table Constraint Style):**

```
CREATE TABLE PRODUCT (  
    P_CODE          VARCHAR(10),  
    P_DESCRIPT     VARCHAR(35)    NOT NULL,  
    P_INDATE       DATE           NOT NULL,  
    P_QOH          SMALLINT       NOT NULL,  
    P_MIN          SMALLINT       NOT NULL,  
    P_PRICE        NUMBER(8,2)    NOT NULL,  
    P_DISCOUNT    NUMBER(5,2)    NOT NULL,  
    V_CODE         NULL,  
  
    PRIMARY KEY (P_CODE),  
    FOREIGN KEY (V_CODE) REFERENCES VENDOR (V_CODE),  
    CHECK (P_QOH >=0 AND P_QOH <= 9999),  
    CHECK (P_MIN >=0 AND P_MIN <= 9999),  
    CHECK (P_PRICE >=0.00 AND P_PRICE <= 9999.00),  
    CHECK (P_DISCOUNT >=0.00 AND P_DISCOUNT <=0.20));
```

❑ **View the structure of the PRODUCT table.**

```
DESCRIBE PRODUCT;  
DESC PRODUCT;
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default
PRODUCT	P_CODE	VARCHAR2	10	-	-	1	-	-
	P_DESCRIPT	VARCHAR2	35	-	-	-	-	-
	P_INDATE	DATE	7	-	-	-	-	-
	P_QOH	NUMBER	22	-	0	-	-	-
	P_MIN	NUMBER	22	-	0	-	-	-
	P_PRICE	NUMBER	-	8	2	-	-	-
	P_DISCOUNT	NUMBER	-	5	2	-	-	-
	V_CODE	NUMBER	22	-	0	-	✓	-

2. SQL DDL: Modifying Database Tables

- ❑ All changes to a table's structure are performed using the **ALTER TABLE** command.
- ❑ Common **ALTER** operations:
 - **ADD** – Adds a new column to the table.
 - **MODIFY** – Changes the characteristics of an existing column (e.g., data type, size).
 - **DROP** – Deletes a column from the table.
- ❑ Other uses of **ALTER TABLE**:
 - **Add** table-level constraints (e.g., **PRIMARY KEY**, **FOREIGN KEY**).
 - **Remove** table-level constraints.

❑ Syntax of the ALTER TABLE Statement:

```
ALTER TABLE table_name
(
    ADD column_name data_type [column_constraint]
|  MODIFY column_name new_data_type
        [column_constraint]
|  DROP COLUMN column_name
|  ADD CONSTRAINT constraint_name constraint_type
        (column_name)
|  DROP CONSTRAINT constraint_name
);
```

- **Explanation of Syntax:**

- **table_name** – Name of the table to be modified.
- **ADD** – Add a column or table-level constraint.
- **MODIFY** – Change an existing column's data type or constraints.
- **DROP COLUMN** – Remove a column from the table.
- **ADD/DROP CONSTRAINT** – Add or remove primary key, foreign key, unique, check, or other constraints.

Example 1: Modifying a Column in an Existing Table

❑ Query

Increase the size of the V_NAME column in the VENDOR table from a variable-length character field of 35 characters to 50 characters.

❑ SQL

```
ALTER TABLE VENDOR  
MODIFY V_NAME VARCHAR(50);
```

❑ View the structure of the VENDOR table:

```
DESCRIBE VENDOR;
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default
VENDOR	V_CODE	NUMBER	22	-	0	1	-	-
	V_NAME	VARCHAR2	50	-	-	-	-	-
	V_CONTACT	VARCHAR2	25	-	-	-	-	-
	V_AREACODE	CHAR	3	-	-	-	-	-
	V_PHONE	CHAR	8	-	-	-	-	-
	V_STATE	CHAR	2	-	-	-	-	-
	V_ORDER	CHAR	1	-	-	-	-	-

Example 2: Modifying Multiple Columns in an Existing Table

❑ Query

Updates the VENDOR table simultaneously by changing the V_CONTACT column to a fixed length of 25 characters and adding a UNIQUE constraint to the V_PHONE column to ensure that all phone numbers are distinct.

❑ SQL

```
ALTER TABLE VENDOR  
MODIFY (V_CONTACT CHAR(25) ,  
        V_PHONE UNIQUE) ;
```

❑ **View the structure of the VENDOR table:**

```
DESCRIBE VENDOR;
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
VENDOR	V_CODE	NUMBER	22	-	0	1	-	-	-
	V_NAME	VARCHAR2	50	-	-	-	-	-	-
	V_CONTACT	CHAR	25	-	-	-	-	-	-
	V_AREACODE	CHAR	3	-	-	-	-	-	-
	V_PHONE	CHAR	8	-	-	-	-	-	-
	V_STATE	CHAR	2	-	-	-	-	-	-
	V_ORDER	CHAR	1	-	-	-	-	-	-

-
- ❑ Use **DESCRIBE VENDOR;** (or **DESC VENDOR;**) to check column names, data types, and NULL constraints.
 - ❑ Limitation – DESCRIBE **does not show** UNIQUE constraints.
 - ❑ To verify constraints such as UNIQUE, query the **system catalog**:

```
SELECT *  
FROM   USER_CONSTRAINTS  
WHERE  TABLE_NAME = 'VENDOR';
```

- ❑ This **lists all constraints** defined on the VENDOR table, including PRIMARY KEY, UNIQUE, FOREIGN KEY, and CHECK.

-
- ❑ To retrieve **only the UNIQUE constraints** defined on the **VENDOR** table, use the following query:

```
SELECT *  
FROM   USER_CONSTRAINTS  
WHERE  TABLE_NAME = 'VENDOR'  
AND   CONSTRAINT_TYPE = 'U';
```

CONSTRAINT_NAME	CONSTRAINT_TYPE	TABLE_NAME
SYS_C00185624102	U	VENDOR

- ❑ **CONSTRAINT_TYPE = 'U'** – Filters only **UNIQUE constraints** (U stands for UNIQUE).

Example 3: Adding a Column to an Existing Table

❑ Query

Add a new column V_FAX to the VENDOR table. The column is of type NUMBER, allows up to 13 digits, and can contain empty values.

❑ SQL

```
ALTER TABLE VENDOR  
ADD V_FAX NUMBER(13) NULL;
```

❑ View the structure of the VENDOR table:

```
DESCRIBE VENDOR;
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
VENDOR	V_CODE	NUMBER	22	-	0	1	-	-	-
	V_NAME	VARCHAR2	50	-	-	-	-	-	-
	V_CONTACT	CHAR	25	-	-	-	-	-	-
	V_AREACODE	CHAR	3	-	-	-	-	-	-
	V_PHONE	CHAR	8	-	-	-	-	-	-
	V_STATE	CHAR	2	-	-	-	-	-	-
	V_ORDER	CHAR	1	-	-	-	-	-	-
	V_FAX	NUMBER	-	13	0	-	✓	-	-

Example 4: Adding Multiple Columns to an Existing Table

❑ Query

Add two new columns to the VENDOR table:

1. V_EMAIL – Variable-length text up to 50 characters, cannot be NULL, and must be unique.
2. V_WEBSITE – Variable-length text up to 100 characters, must be unique but can be NULL.

❑ SQL

```
ALTER TABLE VENDOR  
ADD (V_EMAIL VARCHAR(50) NOT NULL UNIQUE,  
      V_WEBSITE VARCHAR(100) NULL UNIQUE);
```

-
- ❑ View the updated table structure using **DESCRIBE VENDOR;** (or **DESC VENDOR;**) and by querying the system catalog.

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default
VENDOR	V_CODE	NUMBER	22	-	0	1	-	-
	V_NAME	VARCHAR2	50	-	-	-	-	-
	V_CONTACT	CHAR	25	-	-	-	-	-
	V_AREACODE	CHAR	3	-	-	-	-	-
	V_PHONE	CHAR	8	-	-	-	-	-
	V_STATE	CHAR	2	-	-	-	-	-
	V_ORDER	CHAR	1	-	-	-	-	-
	V_FAX	NUMBER	-	13	0	-	✓	-
	V_EMAIL	VARCHAR2	50	-	-	-	-	-
	V_WEBSITE	VARCHAR2	100	-	-	-	✓	-

Example 5: Removing a Column from an Existing Table

❑ Query

Removes the V_FAX column from the VENDOR table.

❑ SQL: Option 1

```
ALTER TABLE VENDOR  
DROP COLUMN V_FAX;
```

❑ SQL: Option 2

```
ALTER TABLE VENDOR  
DROP (V_FAX) ;
```

❑ View the structure of the VENDOR table:

```
DESCRIBE VENDOR;
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
VENDOR	V_CODE	NUMBER	22	-	0	1	-	-	-
	V_NAME	VARCHAR2	50	-	-	-	-	-	-
	V_CONTACT	CHAR	25	-	-	-	-	-	-
	V_AREACODE	CHAR	3	-	-	-	-	-	-
	V_PHONE	CHAR	8	-	-	-	-	-	-
	V_STATE	CHAR	2	-	-	-	-	-	-
	V_ORDER	CHAR	1	-	-	-	-	-	-
	V_EMAIL	VARCHAR2	50	-	-	-	-	-	-
	V_WEBSITE	VARCHAR2	100	-	-	-	✓	-	-

❑ The V_FAX column has been removed.

Example 6: Dropping Multiple Columns from an Existing Table

❑ Query

Removes the V_EMAIL and V_WEBSITE columns from the VENDOR table.

❑ SQL

```
ALTER TABLE VENDOR  
DROP (V_EMAIL, V_WEBSITE) ;
```

❑ **View the structure of the VENDOR table:**

```
DESCRIBE VENDOR;
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
VENDOR	V_CODE	NUMBER	22	-	0	1	-	-	-
	V_NAME	VARCHAR2	50	-	-	-	-	-	-
	V_CONTACT	CHAR	25	-	-	-	-	-	-
	V_AREACODE	CHAR	3	-	-	-	-	-	-
	V_PHONE	CHAR	8	-	-	-	-	-	-
	V_STATE	CHAR	2	-	-	-	-	-	-
	V_ORDER	CHAR	1	-	-	-	-	-	-

- ❑ The V_EMAIL and V_WEBSITE columns **have been removed** as expected.

3. SQL DDL: Dropping Database Tables

❑ Syntax of the DROP TABLE Statement:

```
DROP TABLE table_name;
```

❑ Explanation of Syntax:

- **DROP TABLE** – Command used to **permanently delete** a table from the database.
- **table_name** – The **name of the table** you want to delete.
- **Effect:**
 - Deletes the table **structure** and all the **data** stored in it.
 - **Cannot be rolled back** in most RDBMSs once executed.

❑ **Dropping Tables with Relationships:**

- A table can only be dropped if it is **not the “one” side** of any foreign key relationship.
- If the table is **referenced by** a foreign key in another table (the “many” side), attempting to drop it will cause the RDBMS to **generate an error**.
- This **prevents foreign key integrity violations**, ensuring that related data in other tables is **not left orphaned**.
- **Example Scenario:**
 - VENDOR table is **referenced by** PRODUCT through V_CODE.
 - Dropping VENDOR **without first removing or altering** the foreign key in PRODUCT will **result in an error**.

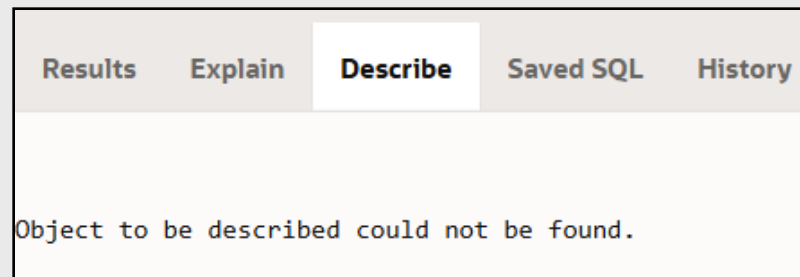
❑ Query

Completely deletes the PRODUCT table, including all its data and schema, from the database.

❑ SQL

```
DROP TABLE PRODUCT;
```

❑ View the updated table structure using DESCRIBE PRODUCT ;



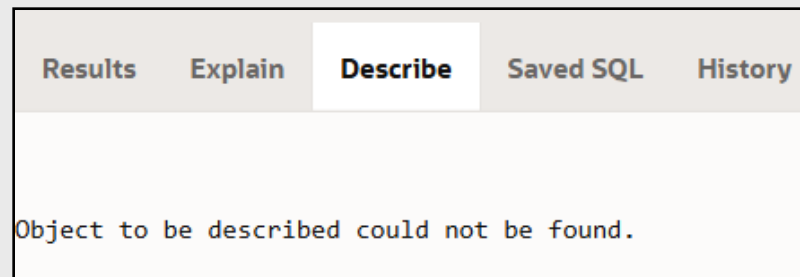
❑ Query

Completely deletes the VENDOR table, including all its data and schema, from the database.

❑ SQL

```
DROP TABLE VENDOR;
```

❑ View the updated table structure using DESCRIBE VENDOR;



Part 2: SQL Data Manipulation Language (DML)

- ❑ SQL Data Manipulation Commands (DML) refer to the set of SQL statements used to **retrieve**, **insert**, **update**, and **delete** data stored in a database.
- ❑ The **main DML commands** are:
 - **INSERT** – Adds new rows of data into a table.
 - **UPDATE** – Modifies existing data in a table.
 - **DELETE** – Removes existing rows from a table.
 - **SELECT** – Retrieves data from one or more tables (will be covered in Lectures 8 to 11)
- ❑ In short:

DML = Commands for handling and manipulating the data inside tables (**not the structure**).

❑ SQL Data Manipulation Commands:

**TABLE
5.2**

SQL Data Manipulation Commands

COMMAND OR OPTION	DESCRIPTION
INSERT	Inserts row(s) into a table
SELECT	Selects attributes from rows in one or more tables or views
WHERE	Restricts the selection of rows based on a conditional expression
GROUP BY	Groups the selected rows based on one or more attributes
HAVING	Restricts the selection of grouped rows based on a condition
ORDER BY	Orders the selected rows based on one or more attributes
UPDATE	Modifies an attribute's values in one or more table's rows
DELETE	Deletes one or more rows from a table
COMMIT	Permanently saves data changes
ROLLBACK	Restores data to their original values
Comparison operators	
=, <, >, <=, >=, <>	Used in conditional expressions
Logical operators	
AND/OR/NOT	Used in conditional expressions
Special operators	
BETWEEN	Checks whether an attribute value is within a range
IS NULL	Checks whether an attribute value is null
LIKE	Checks whether an attribute value matches a given string pattern
IN	Checks whether an attribute value matches any value within a value list
EXISTS	Checks whether a subquery returns any rows
DISTINCT	Limits values to unique values
Aggregate functions	
COUNT	Used with SELECT to return mathematical summaries on columns
COUNT	Returns the number of rows with non-null values for a given column
MIN	Returns the minimum attribute value found in a given column
MAX	Returns the maximum attribute value found in a given column
SUM	Returns the sum of all values for a given column
AVG	Returns the average of all values for a given column

-
- ❑ The **VENDOR** and **PRODUCT** tables will be **re-created** to illustrate the use of SQL DML commands.

```
CREATE TABLE VENDOR
(
  V_CODE          INTEGER,
  V_NAME          VARCHAR(35)      NOT NULL,
  V_CONTACT       VARCHAR(25)      NOT NULL,
  V_AREACODE      CHAR(3)          NOT NULL,
  V_PHONE         CHAR(8)          NOT NULL,
  V_STATE         CHAR(2)          NOT NULL,
  V_ORDER         CHAR(1)          NOT NULL,

  PRIMARY KEY (V_CODE),
  CHECK (V_CODE >= 1000 AND V_CODE <= 9999),
  CHECK (V_ORDER = 'Y' OR V_ORDER = 'N')
);
```

```
CREATE TABLE PRODUCT
(
    P_CODE          VARCHAR(10),
    P_DESCRIPT     VARCHAR(35)    NOT NULL,
    P_INDATE       DATE           NOT NULL,
    P_QOH          SMALLINT       NOT NULL,
    P_MIN          SMALLINT       NOT NULL,
    P_PRICE        NUMBER(8,2)    NOT NULL,
    P_DISCOUNT    NUMBER(5,2)    NOT NULL,
    V_CODE         NULL,

    PRIMARY KEY (P_CODE),
    FOREIGN KEY (V_CODE) REFERENCES VENDOR (V_CODE),
    CHECK (P_QOH >=0 AND P_QOH <= 9999),
    CHECK (P_MIN >=0 AND P_MIN <= 9999),
    CHECK (P_PRICE >=0.00 AND P_PRICE <= 9999.00),
    CHECK (P_DISCOUNT>=0.00 AND P_DISCOUNT <=0.20)
);
```

1. SQL DML: Inserting New Data

❑ Syntax of the SQL INSERT Statement:

- **Inserting values into all columns:** Must provide values for all columns in the same order as the table.

```
INSERT INTO table_name  
VALUES (value1, value2, value3, ...);
```

- **Inserting values into specific columns:** Only the listed columns will be filled; the rest will take their default values or remain NULL (if allowed).

```
INSERT INTO table_name  
(column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

❑ Guidelines for Entering Values in SQL:

- **Row contents** are enclosed within parentheses ().
- **Character and date** values must be enclosed in apostrophes ' '.
- **Numeric values** are written without apostrophes.
- **Attribute values** are separated by commas ,.
- A **value** must be provided for each column (unless defaults are defined).
- Use **NULL** to represent unknown or missing values.

Example 1: Inserting Data into All Columns (Specifying the Column List)

❑ Query

Insert a new vendor with the following details:

- Vendor Code: "1001"
- Vendor Name: "Tech Supplies Sdn Bhd"
- Contact Person: "Muhammad Ammar Bin Sani"
- Area Code: "03"
- Phone Number: "45678901"
- State: "KL"
- Active Order Status: "N"

❑ SQL

```
INSERT INTO VENDOR (V_CODE, V_NAME, V_CONTACT,  
V_AREACODE, V_PHONE, V_STATE, V_ORDER)  
VALUES (1001, 'Tech Supplies Sdn Bhd', 'Muhammad Ammar  
Bin Sani', '03', '45678901', 'KL', 'N');
```

❑ View the table data using **SELECT * FROM VENDOR;**

V_CODE	V_NAME	V_CONTACT	V_AREACODE	V_PHONE	V_STATE	V_ORDER
1001	Tech Supplies Sdn Bhd	Muhammad Ammar Bin Sani	03	45678901	KL	N

Example 2: Inserting Data into All Columns (Without Specifying the Column List)

❑ Query

Insert a new vendor with the following details:

- Vendor Code: "1002"
- Vendor Name: "OfficeMart Sdn Bhd"
- Contact Person: "Chuah Ming Xuan"
- Area Code: "04"
- Phone Number: "78965432"
- State: "PN"
- Active Order Status: "Y"

❑ SQL

```
INSERT INTO VENDOR  
VALUES (1002, 'OfficeMart Sdn Bhd', 'Chuah Ming Xuan',  
'04', '78965432', 'PN', 'Y');
```

Note: When the column list is omitted, you **must provide values for every column** in the correct order as defined in the table structure. If you skip a value or mismatch the order, the **DBMS will return an error**.

❑ View the table data using **SELECT * FROM VENDOR;**

V_CODE	V_NAME	V_CONTACT	V_AREACODE	V_PHONE	V_STATE	V_ORDER
1001	Tech Supplies Sdn Bhd	Muhammad Ammar Bin Sani	03	45678901	KL	N
1002	OfficeMart Sdn Bhd	Chuah Ming Xuan	04	78965432	PN	Y

Example 3: Inserting Data into All Columns (Specifying a Different Column Order)

❑ Query

Insert a new vendor with the following details, but list the columns in a different order:

- Vendor Code: "1003"
- Vendor Name: "Digital World Sdn Bhd"
- Contact Person: "Eisshvaran A/L Saravanan"
- Area Code: "06"
- Phone Number: "33445566"
- State: "ML"
- Active Order Status: "Y"

❑ SQL

```
INSERT INTO VENDOR
(V_NAME, V_CONTACT, V_PHONE, V_AREACODE, V_STATE, V_ORDER, V_CODE)
VALUES ('Digital World Sdn Bhd', 'Eisshvaran A/L Saravanan',
'33445566', '06', 'ML', 'Y', 1003);
```

Note: As long as you specify the **column names explicitly**, the order of the columns in the INSERT statement **can differ** from the table definition. The **values must match** the order of the listed columns, not the original table structure.

❑ View the table data using **SELECT * FROM VENDOR;**

V_CODE	V_NAME	V_CONTACT	V_AREACODE	V_PHONE	V_STATE	V_ORDER
1001	Tech Supplies Sdn Bhd	Muhammad Ammar Bin Sani	03	45678901	KL	N
1002	OfficeMart Sdn Bhd	Chuah Ming Xuan	04	78965432	PN	Y
1003	Digital World Sdn Bhd	Eisshvaran A/L Saravanan	06	33445566	ML	Y

Example 4: Inserting Data Containing Foreign Key Values

❑ Query

Insert a new product with the following details, ensuring the V_CODE exists in the VENDOR table:

- Product Code: "P1001"
- Description: "Wireless Mouse"
- Date Added: "26-SEP-2025"
- Quantity on Hand: "150"
- Minimum Stock: "20"
- Price: "59.90"
- Discount: "0.10"
- Vendor Code: "1001" (must already exist in the VENDOR table)

❑ SQL

```
INSERT INTO PRODUCT(P_CODE, P_DESCRIPT, P_INDATE, P_QOH, P_MIN,  
P_PRICE, P_DISCOUNT, V_CODE)  
VALUES ('P1001', 'Wireless Mouse', '09-26-2025', 150, 20, 59.90, 0.10,  
1001);
```

Note: The value 1001 for V_CODE **must already exist** in the VENDOR table (VENDOR.V_CODE = 1001). If you try to insert a V_CODE that **does not exist**, the **DBMS will reject the insertion** due to referential integrity.

❑ View the table data using **SELECT * FROM PRODUCT;**

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
P1001	Wireless Mouse	9/26/2025	150	20	59.9	.1	1001

Example 5: Inserting Data into All Mandatory Columns (Using empty string (' ') or NULL for Optional Columns)

❑ Query

Insert a new product with the following details:

- Product Code: "P1002"
- Description: "USB-C Cable"
- Date Added: "26-SEP-2025"
- Quantity on Hand: "200"
- Minimum Stock: "30"
- Price: "19.90"
- Discount: "0.05"
- Vendor Code: not applicable, because the product is produced in-house, so set it as NULL.

❑ SQL

```
INSERT INTO PRODUCT (P_CODE, P_DESCRIPT, P_INDATE,  
P_QOH, P_MIN, P_PRICE, P_DISCOUNT, V_CODE)  
VALUES ('P1002', 'USB-C Cable', '09-26-2025', 200, 30,  
19.90, 0.05, '');
```

Note: An **empty string** (' ') may be replaced with **NULL**.

❑ View the table data using **SELECT * FROM PRODUCT;**

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
P1001	Wireless Mouse	9/26/2025	150	20	59.9	.1	1001
P1002	USB-C Cable	9/26/2025	200	30	19.9	.05	-

Example 6: Inserting Data into All Mandatory Columns (Omitting Optional Columns)

❑ Query

Insert a new product with the following details:

- Product Code: "P1003"
- Description: "Bluetooth Speaker"
- Date Added: "27-SEP-2025"
- Quantity on Hand: "75"
- Minimum Stock: "10"
- Price: "120.00"
- Discount: "0.15"
- Vendor Code: not specified (optional column, omitted)

❑ SQL

```
INSERT INTO PRODUCT (P_CODE, P_DESCRIPT, P_INDATE, P_QOH,  
P_MIN, P_PRICE, P_DISCOUNT)  
VALUES ('P1003', 'Bluetooth Speaker', '09-27-2025', 75, 10,  
120.00, 0.15);
```

Note: Since V_CODE is **optional (nullable)**, it can be **omitted from both the column list and the VALUES list**. The **DBMS will insert NULL** into V_CODE automatically.

❑ View the table data using **SELECT * FROM PRODUCT;**

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
P1001	Wireless Mouse	9/26/2025	150	20	59.9	.1	1001
P1002	USB-C Cable	9/26/2025	200	30	19.9	.05	-
P1003	Bluetooth Speaker	9/27/2025	75	10	120	.15	-

2. SQL DML: Updating Existing Data

❑ Syntax of the SQL UPDATE Statement:

```
UPDATE table_name  
SET      column_name = value | expression  
        | DEFAULT | NULL  
WHERE search_condition
```

❑ Explanation of Syntax:

- **UPDATE table_name** – specifies the table you want to update.
- **SET column_name = ...**
 - Identifies the column(s) to be updated.
 - The new value can be: value, expression (a calculation or function), DEFAULT, NULL
- **WHERE search_condition** – specifies which rows to update (very important, otherwise all rows will be updated).

Example 1: Updating All Rows

❑ Query

Update the P_DISCOUNT value for all products to “0.20”.

❑ SQL

```
UPDATE PRODUCT  
SET      P_DISCOUNT = 0.20;
```

❑ **Note:**

- Because there is **no WHERE clause**, the update **affects every row** in the PRODUCT table.
- Use this type of statement **with caution**, as it will **overwrite** the existing discount value **for all products**.

❑ View the table data using **SELECT * FROM PRODUCT;**

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
P1001	Wireless Mouse	9/26/2025	150	20	59.9	.2	1001
P1002	USB-C Cable	9/26/2025	200	30	19.9	.2	-
P1003	Bluetooth Speaker	9/27/2025	75	10	120	.2	-

Example 2: Updating Specific Rows

❑ Query

Update the P_DESCRIPT (product description) of the product with code "P1002" to "High-Speed USB-C Cable".

❑ SQL

```
UPDATE PRODUCT
SET      P_DESCRIPT = 'High-Speed USB-C Cable'
WHERE    P_CODE = 'P1002';
```

❑ **Note:**

- The **WHERE** clause ensures that **only** the product with P_CODE = 'P1002' is updated.
- Always include a **WHERE** condition when updating specific rows to **avoid changing the entire table**.

❑ View the table data using **SELECT * FROM PRODUCT;**

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
P1001	Wireless Mouse	9/26/2025	150	20	59.9	.2	1001
P1002	High-Speed USB-C Cable	9/26/2025	200	30	19.9	.2	-
P1003	Bluetooth Speaker	9/27/2025	75	10	120	.2	-

Example 3: Updating Specific Rows Using an Expression

❑ Query

Reduce the quantity on hand (P_QOH) of product "P1002" by 10 units.

❑ SQL

```
UPDATE PRODUCT
SET      P_QOH = P_QOH - 10
WHERE    P_CODE = 'P1002';
```

❑ **Note:**

- The expression **P_QOH - 10** updates the column based on its **current value**, reducing the stock dynamically instead of assigning a fixed number.
- The **WHERE clause** ensures that only the product with **P_CODE = 'P1002'** is updated.

❑ View the table data using **SELECT * FROM PRODUCT;**

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
P1001	Wireless Mouse	9/26/2025	150	20	59.9	.2	1001
P1002	High-Speed USB-C Cable	9/26/2025	190	30	19.9	.2	-
P1003	Bluetooth Speaker	9/27/2025	75	10	120	.2	-

Example 4: Updating Multiple Columns

❑ Query

Update the product with code “P1003” by changing its entry date to “September 29, 2025” and setting its price to “79.90”.

❑ SQL

```
UPDATE PRODUCT
SET      P_INDATE = '9/29/2025',
         P_PRICE  = 79.90
WHERE P_CODE = 'P1003';
```

❑ **Note:**

- **Multiple columns** can be updated in a single **UPDATE** statement by separating each assignment with a **comma**.
- The **WHERE clause** ensures that only the row with **P_CODE = 'P1003'** is updated.

❑ View the table data using **SELECT * FROM PRODUCT;**

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
P1001	Wireless Mouse	9/26/2025	150	20	59.9	.2	1001
P1002	High-Speed USB-C Cable	9/26/2025	190	30	19.9	.2	-
P1003	Bluetooth Speaker	9/29/2025	75	10	79.9	.2	-

3. SQL DML: Deleting Existing Data

❑ Syntax of the **SQL DELETE** Statement:

```
DELETE FROM table_name  
WHERE search_condition
```

❑ **Explanation** of Syntax:

- **DELETE FROM** – specifies the table from which rows will be removed.
- **WHERE** – specifies which rows to delete. Without it, all rows in the table will be deleted.

Example 1: Deleting Specific Rows

❑ Query

Delete the product with code “P1003” from the PRODUCT table.

❑ SQL

```
DELETE FROM PRODUCT  
WHERE P_CODE = 'P1003';
```

❑ **Note:**

- The row containing product P1003 is **permanently removed** from the PRODUCT table.
- All **other rows** remain unchanged.

❑ View the table data using **SELECT * FROM PRODUCT;**

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
P1001	Wireless Mouse	9/26/2025	150	20	59.9	.2	1001
P1002	High-Speed USB-C Cable	9/26/2025	190	30	19.9	.2	-

Example 2: Deleting Specific Rows Involving Foreign Key Relationships

❑ Query

Delete the vendor with code “1001” from the VENDOR table.

❑ SQL

```
DELETE FROM VENDOR  
WHERE V_CODE = 1001;
```

❑ Result/Error

```
ORA-02292: integrity constraint (WKSP_NORMASAINA.SYS_C00185670987) violated - child record found
```

Note:

- This **error occurs** because V_CODE = 1001 is **referenced as a foreign key** in the PRODUCT table, product with **P_CODE P1001**.
- The database prevents deletion to **maintain referential integrity**.

❑ Delete product(s) linked to **Vendor “1001”** first.

```
DELETE FROM PRODUCT  
WHERE V_CODE = 1001;
```

-
- ❑ View the table data using **SELECT * FROM PRODUCT;**

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
P1002	High-Speed USB-C Cable	9/26/2025	190	30	19.9	.2	-

Note:

- This **removes** all products **supplied by vendor 1001**.
 - All **other row(s)** remain unchanged.
- ❑ Now the vendor (**V_CODE = 1001**) row can be deleted successfully, since no products are referencing it.

```
DELETE FROM VENDOR
WHERE V_CODE = 1001;
```

-
- ❑ View the table data using **SELECT * FROM VENDOR;**

V_CODE	V_NAME	V_CONTACT	V_AREACODE	V_PHONE	V_STATE	V_ORDER
1002	OfficeMart Sdn Bhd	Chuah Ming Xuan	04	78965432	PN	Y
1003	Digital World Sdn Bhd	Eisshvaran A/L Saravanan	06	33445566	ML	Y

Note:

- **Vendor 1001** was **successfully deleted**.
- **All other rows** remain unchanged.

Example 3: Deleting All Rows

❑ Query

Delete all rows from the PRODUCT table, leaving the table structure intact.

❑ SQL

```
DELETE FROM PRODUCT;
```

❑ **Note:**

- **All rows** in the PRODUCT table are **permanently removed**.
- The DELETE statement **without a WHERE clause** removes all rows from the table.
- The table structure, columns, and constraints **remain unchanged**.

❑ View the table data using **SELECT * FROM PRODUCT;**

Results	Explain	Describe	Saved SQL	History
no data found				

Summary

- ❑ **SQL** is the standard programming language for managing and manipulating **relational databases**.
- ❑ **SQL DDL (Data Definition Language)**
 - Used to **define and modify** database structures.
 - Common **commands**:
 - **CREATE TABLE** – Create a new table.
 - **ALTER TABLE** – Modify an existing table (add, modify, or drop columns/constraints).
 - **DROP TABLE** – Permanently delete a table and its data.
 - Enforces **data integrity** through constraints like **PRIMARY KEY**, **FOREIGN KEY**, **UNIQUE**, **CHECK**, and **NOT NULL**.
 - Changes affect the **table structure**, not just the data.

❑ SQL DML (Data Manipulation Language)

- Used to **manipulate data** within tables.
- Common **commands**:
 - **INSERT** – Add new rows to a table.
 - **UPDATE** – Modify existing data.
 - **DELETE** – Remove rows from a table.
- Changes **affect table contents**, not the structure.
- Can be combined with **WHERE clauses** to target specific rows.